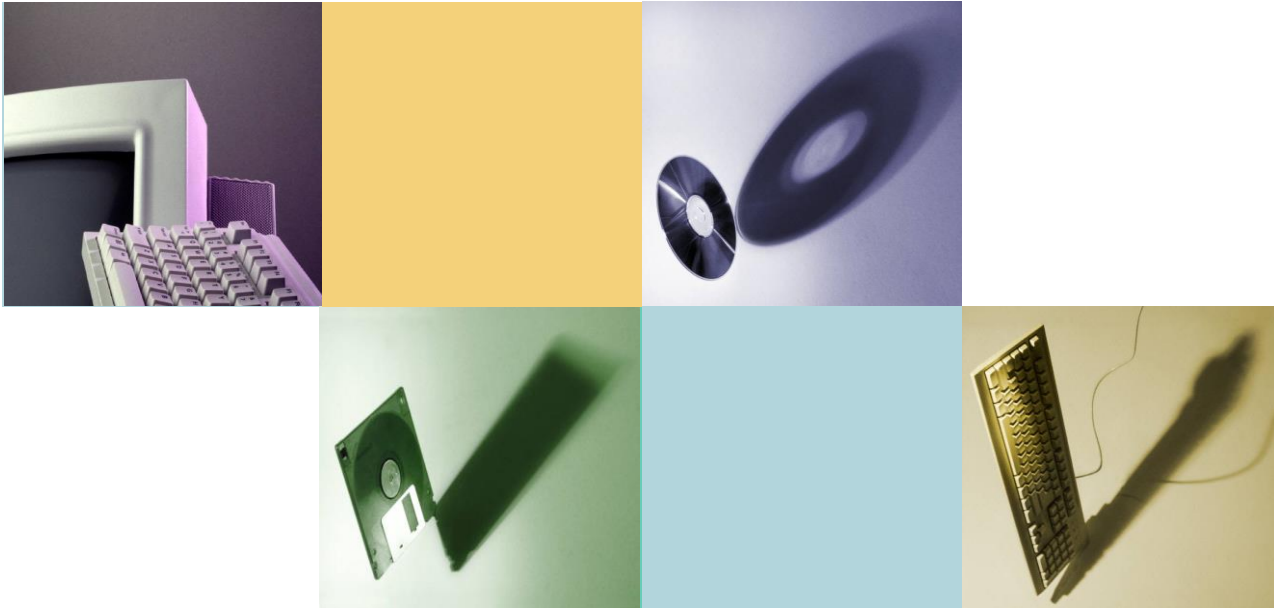
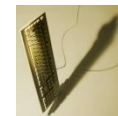


第3章 指令系统



第3章 指令系统

- 概述
- 寻址方式
- 指令系统



3.1 概述

3.1.1 指令的格式

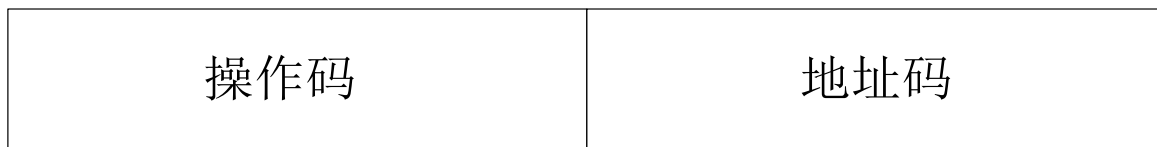


图3-1 80x86指令的格式



3.1 概述

3.1.2 指令的编码格式

前缀	操作码	寻址方式	位移量	立即数
1个字节	1个字节	1个字节	0~2个字节	0~2个字节

图3-2 8086指令的编码格式



3.2 寻址方式

3.2.1 指令的寻址方式

● 顺序寻址

顺序结构的程序，执行时按照其代码编写顺序(内存存储顺序)执行。**CPU**根据**IP**指向的地址取出指令，分析该指令的长度，将**IP**的值加上指令的长度，形成下一条指令的地址，这种寻找下一条指令的方法称为指令的顺序寻址。

● 转移寻址

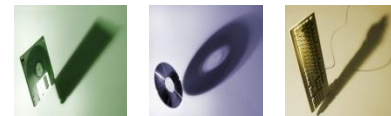
遇到除顺序结构外的其他结构时，下一条指令的地址本条指令(转移指令，过程调用指令，返回指令等)给出的，本条指令执行时，由**CPU**将下一条指令的地址传送至**CS**和**IP**，实现程序的转移，这种寻找下一条指令地址的方法称为指令的转移寻址。



3.2.1 操作数的寻址方式

1.操作数分类。

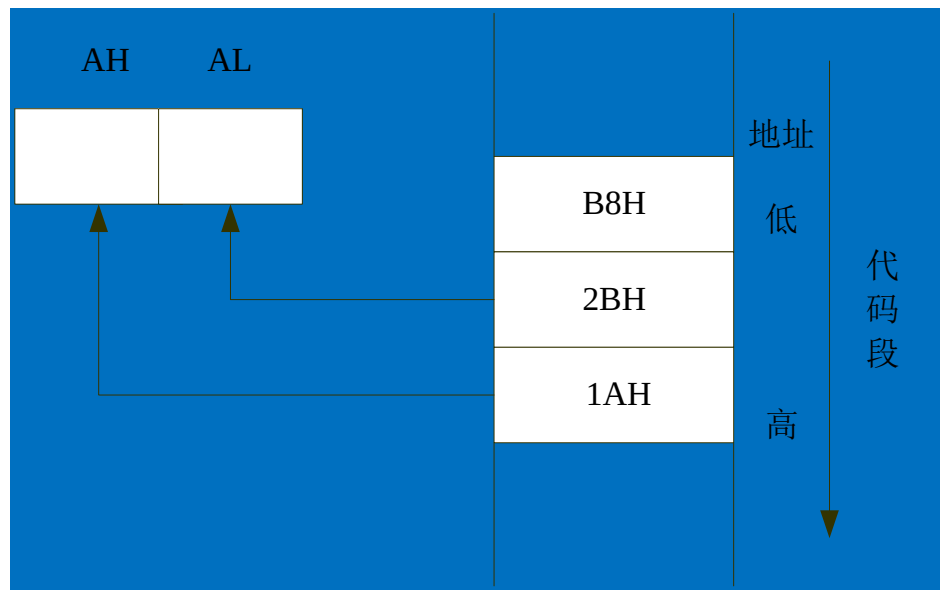
- 源操作数在指令中只参与运算，指令执行后源操作数不发生改变。
- 目的操作数在指令中既要参见运算，一般还用于保存运算的结果，指令执行后目的操作数的值通常会发生改变。



2. 操作数的寻址方式

1) 立即寻址:操作数存放在指令的地址码字段，即操作数是指令的一部分，操作数存放在存储器的代码段，取指令的同时连同操作数一起取至**CPU**。操作数可以是**8位**，**16位**，存放时高位存放在高地址存储单元，低位存放在低地址存储单元。

例：MOV AX, 1A2BH



2) 寄存器寻址(Register Addressing)

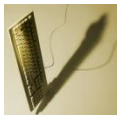
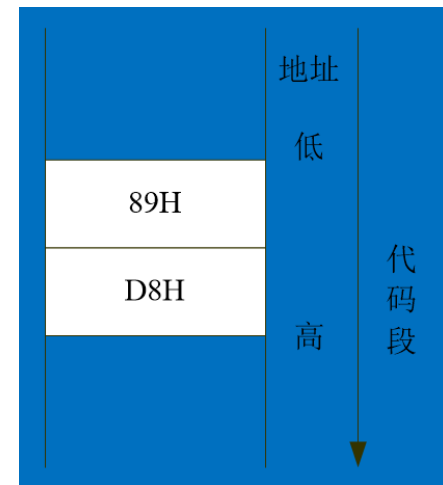
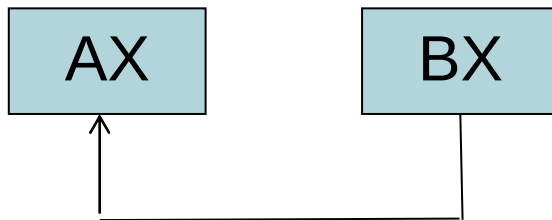
操作数存放在**CPU**内部的寄存器中

8位寄存器:AL, AH, BL, BH, CL, CH, DL, DH,

16位通用寄存器:AX, BX, CX, DX, SI, DI, SP, BP

16位段寄存器:CS, SS, DS, ES。

例: **MOV AX, BX**

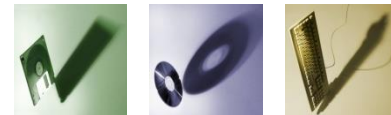


3) 直接寻址(Direct Addressing)

指令的地址码字段给出直接的是操作数在存储器中的**16位偏移地址(EA)**。

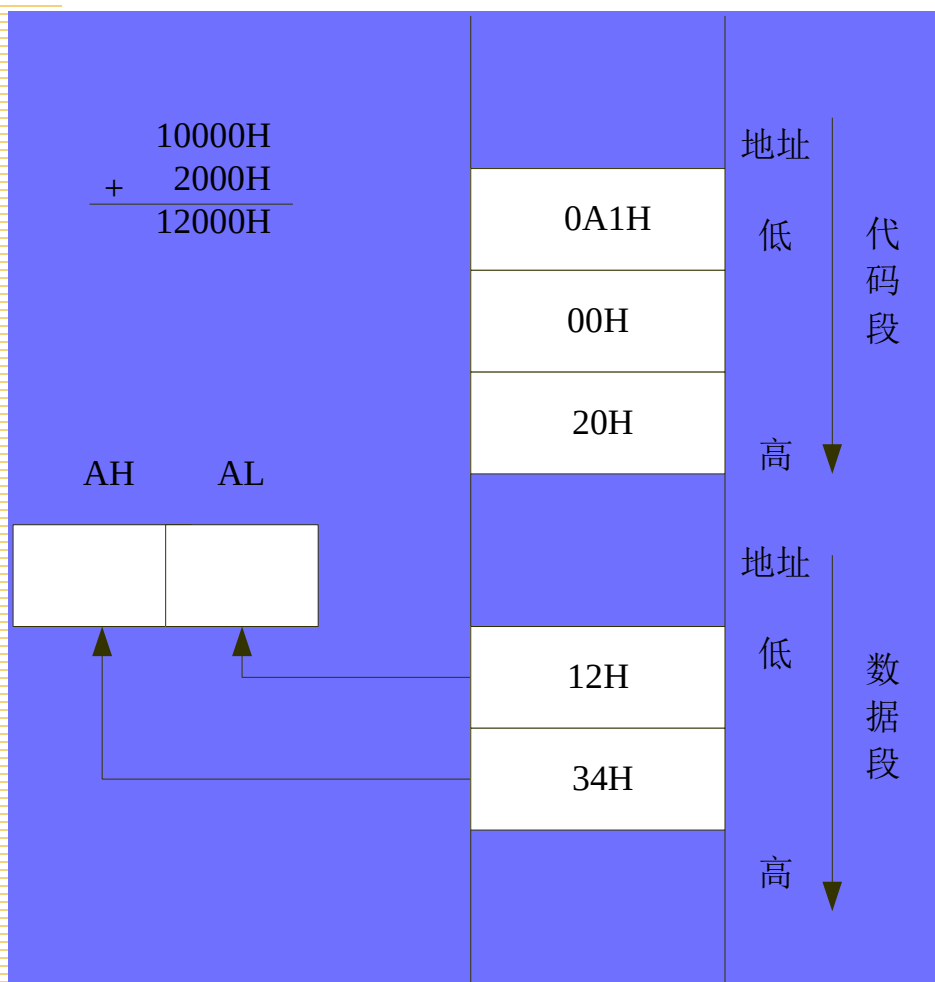
段地址:

- 如果指令前没有加操作数限制前缀，则操作数默认存放在存储器的数据段，操作数的段地址为**DS**的值
- 如果指令前有操作数限制前缀，则操作数的段地址由操作数限制前缀中给定的段寄存器确定。



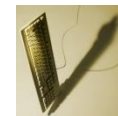
例: MOV AX, [2000H]

假设指令执行前(DS)=1000H, (12000H)=12H, (12001H)=34H。



源操作数的物理地址

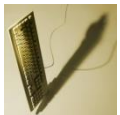
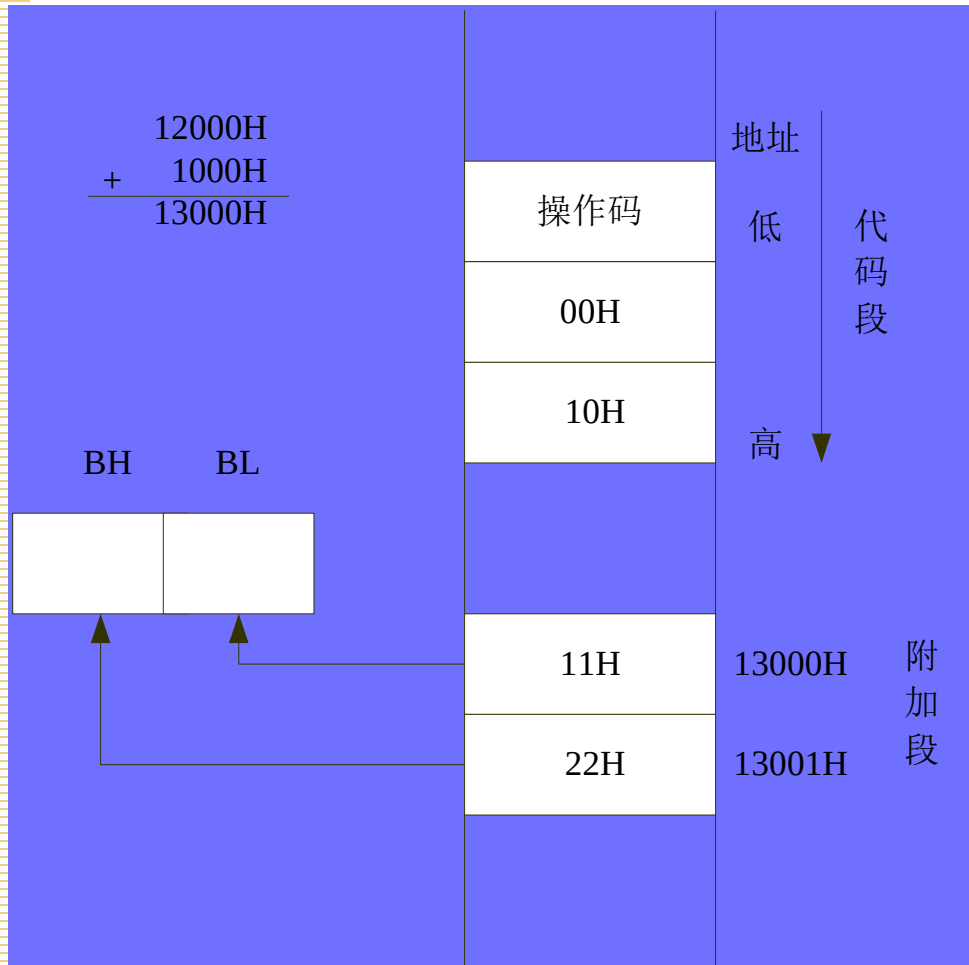
$$1000H \times 10H + 2000H = 12000H$$



例: MOV BX, **ES:[1000H]** ;(ES)=1200H

源操作数的物理地址

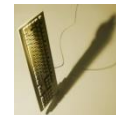
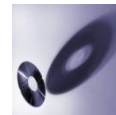
=1200H×10H+1000H=13000H



4) 寄存器间接寻址(Register Indirect Addressing)

操作数存放在存储器中，其偏移地址存放在**16**位寄存器中，
可以存放操作数偏移地址的寄存器有4个**BX**，**BP**，**SI**和**DI**。

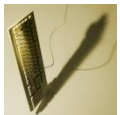
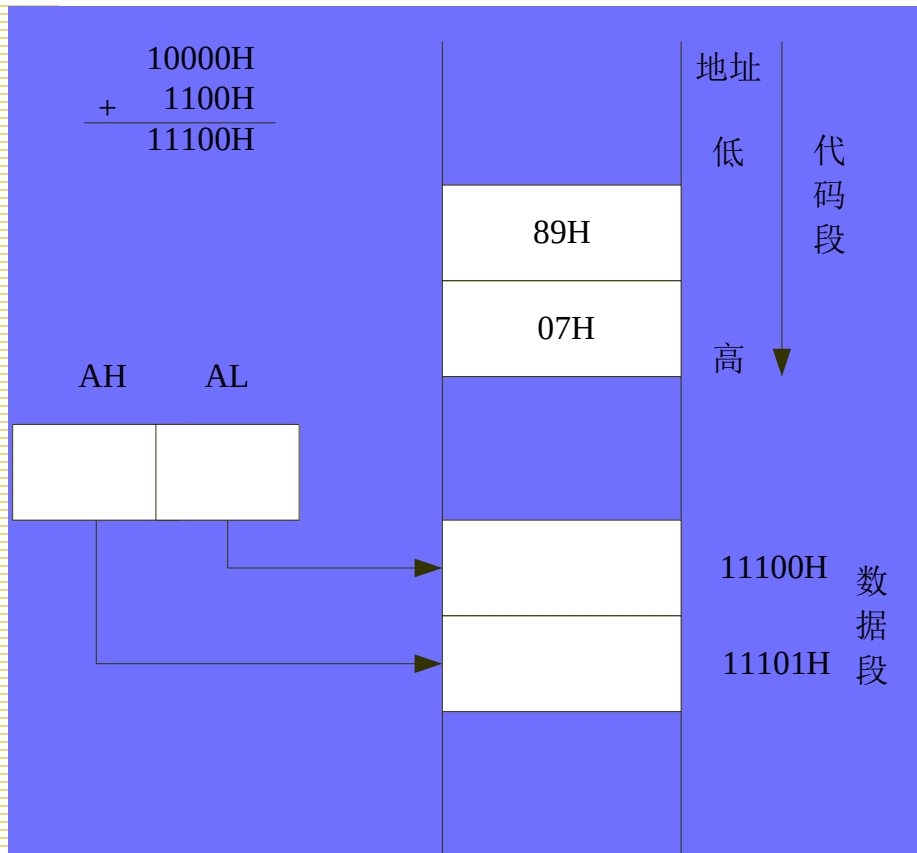
偏移地址	段地址
BX	DS
SI	DS
DI	DS
BP	SS



例: MOV [BX],AX

假设(DS)=1000H, (BX)=1100H。

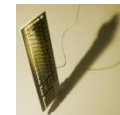
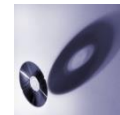
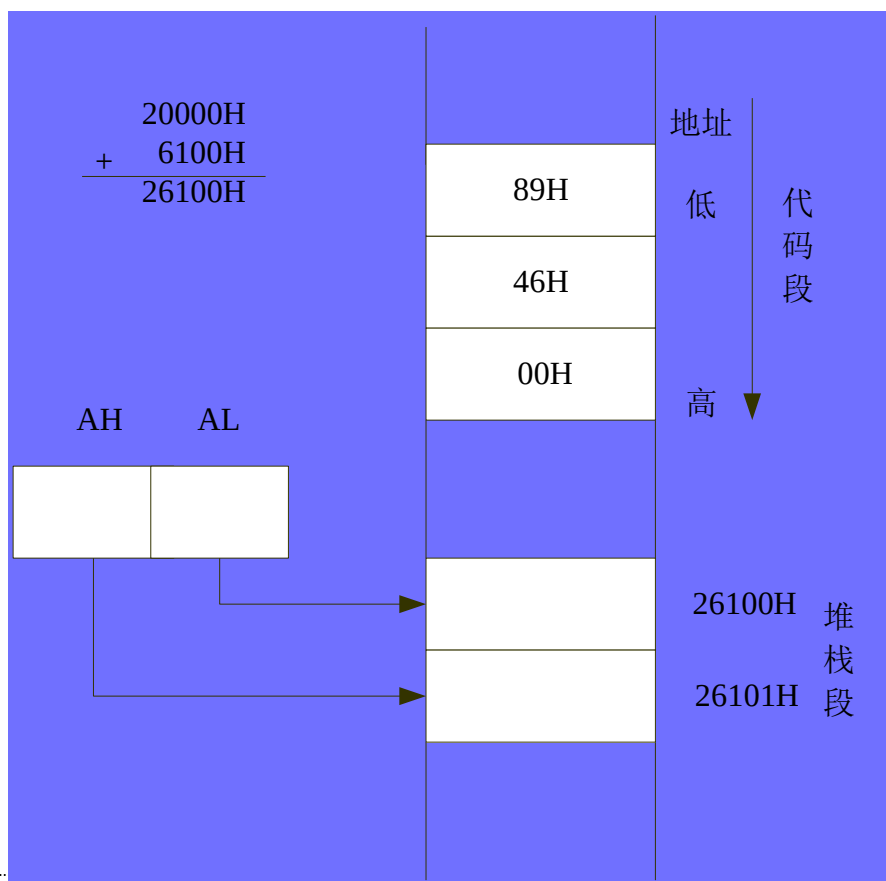
目的操作数的物理地址
 $=1000H \times 10H + 1100H = 11100H$



例: MOV [BP],AX

假设(SS)=2000H, (BP)=6100H。

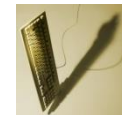
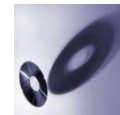
目的操作数的偏移地址是**6100H**,
段地址是**2000H**, 物理地址
=2000H×10H+6100H=26100H



5) 寄存器相对寻址方式(Register Relative Addressing)

操作数存放在存储器中，操作数的偏移地址是基址寄存器或者变址寄存器的内容加上**8位**或者**16位**的位移量。即操作数的有效地址

$$EA = \begin{pmatrix} SI \\ DI \\ BX \\ BP \end{pmatrix} + \begin{pmatrix} 8\text{位disp} \\ 16\text{位disp} \end{pmatrix}$$



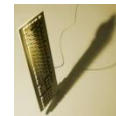
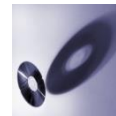
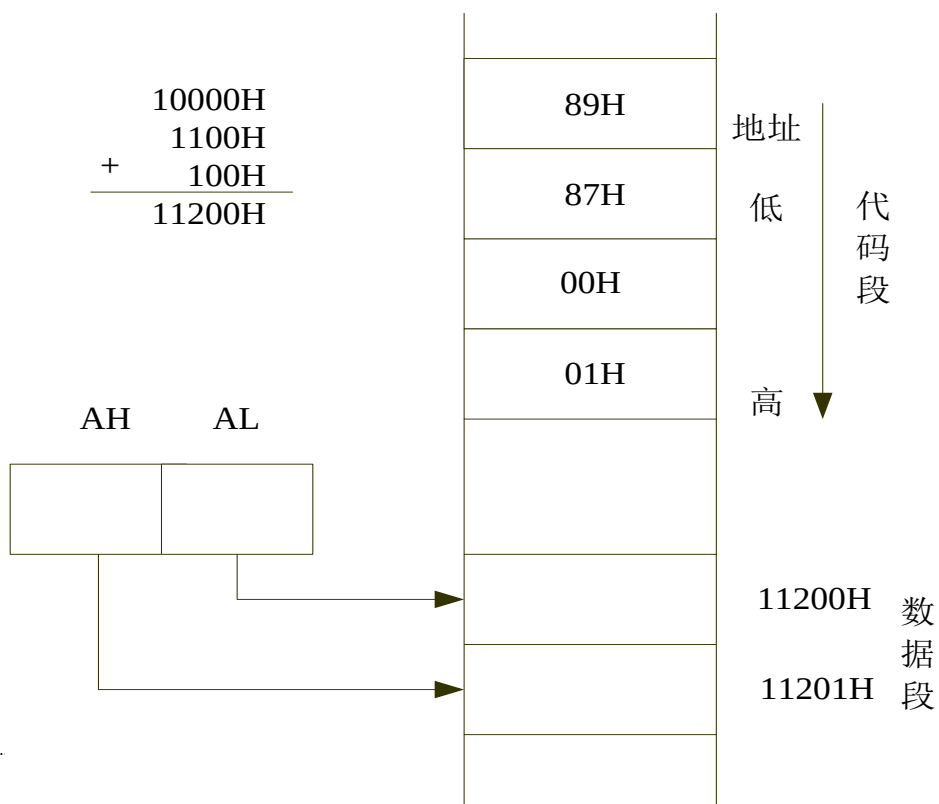
例 **MOV [BX+100H],AX ; (DS)=1000H, (BX)=1100H,**

目的操作数的寻址方式：寄存器相对寻址方式

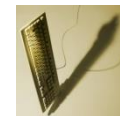
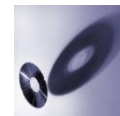
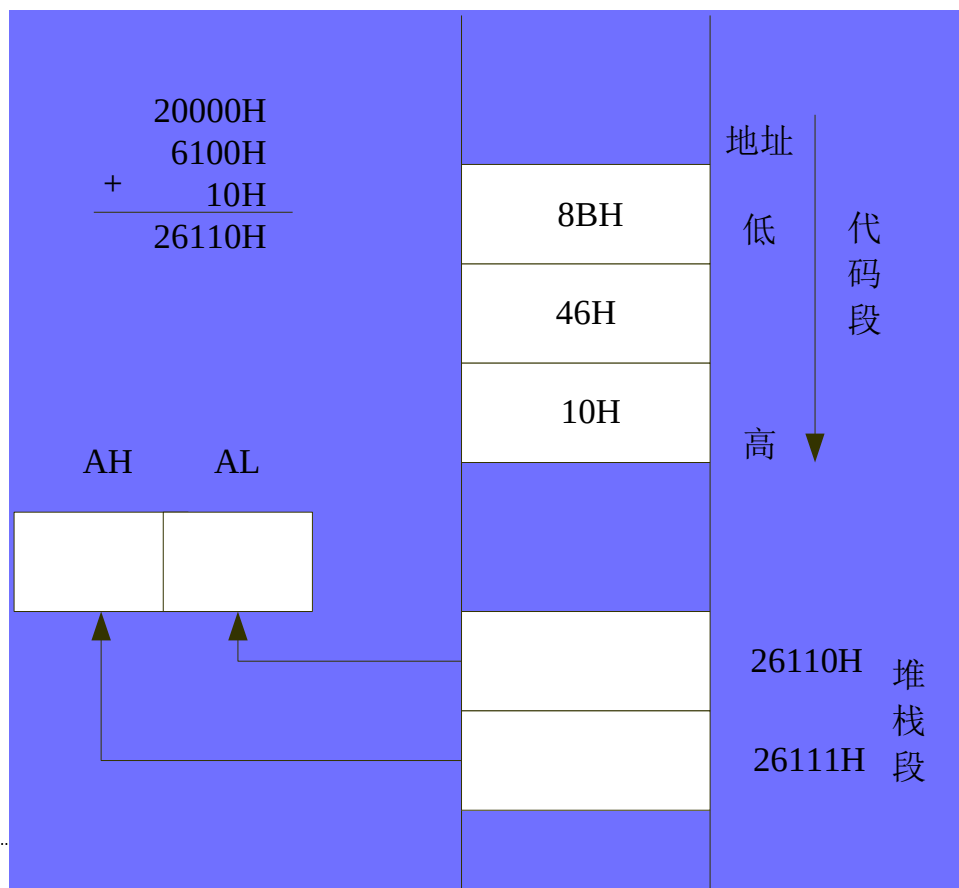
目的操作数的偏移地址=**(BX)+100H=1200H,**

目的操作数的段地址为**DS**的内容**1000H,**

目的操作数的物理地址=**(DS)×10H+(BX)+100H=11200H**

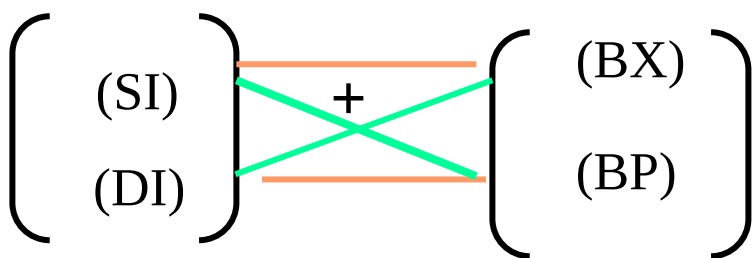


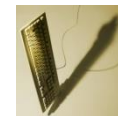
例 **MOV AX,[BP+10H]; (SS)=2000H, (BP)=6100H,**
源操作数：寄存器相对寻址方式
源操作数的偏移地址=**(BP)+10H=6110H,**
源操作数的段地址为**SS的内容2000H,**
源操作数的物理地址=**(SS)×10H+(BP)+10H=26110H**



6) 基址变址寻址(Based Indexed Addressing)

操作数存放在存储器中，操作数的偏移地址是一个基址寄存器的内容和一个变址寄存器的内容之和，即有效地址

$$EA = \begin{pmatrix} (SI) \\ (DI) \end{pmatrix} + \begin{pmatrix} (BX) \\ (BP) \end{pmatrix}$$




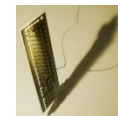
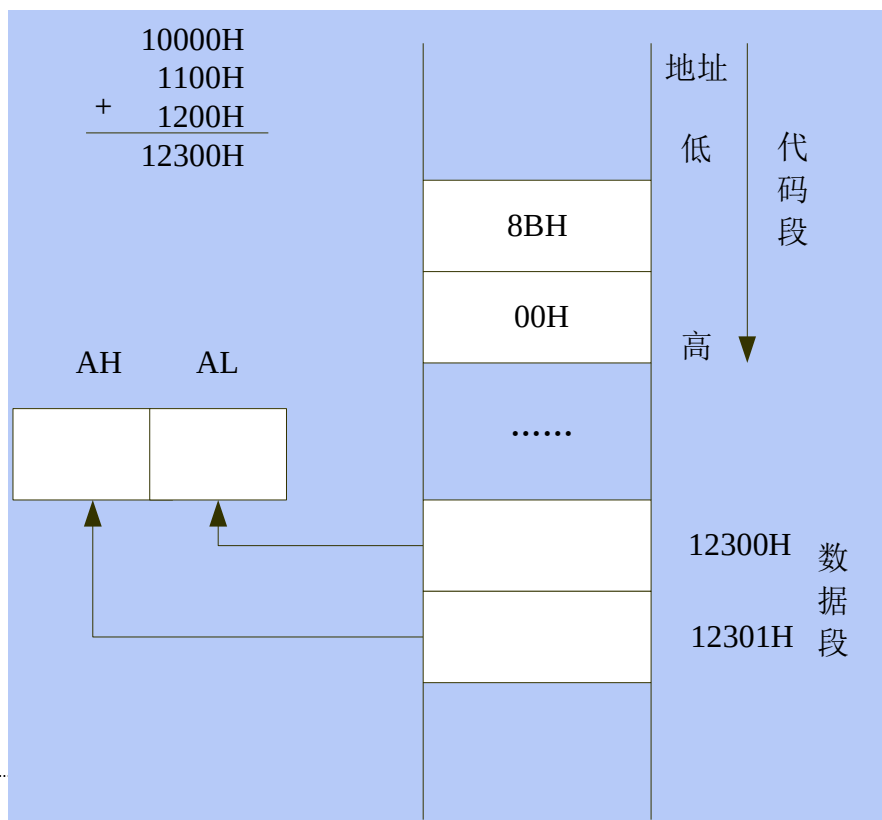
例 **MOV AX,[BX+SI]; (DS)=1000H,(BX)=1100H,(SI)=1200H,**

源操作数:基址变址寻址,

源操作数的偏移地址=**(BX)+(SI)=2300H,**

源操作数的段地址为**DS**的内容**1000H,**

源操作数的物理地址=**(DS)×10H+(BX)+(SI)=12300H。**



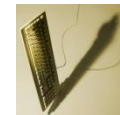
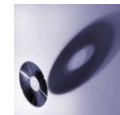
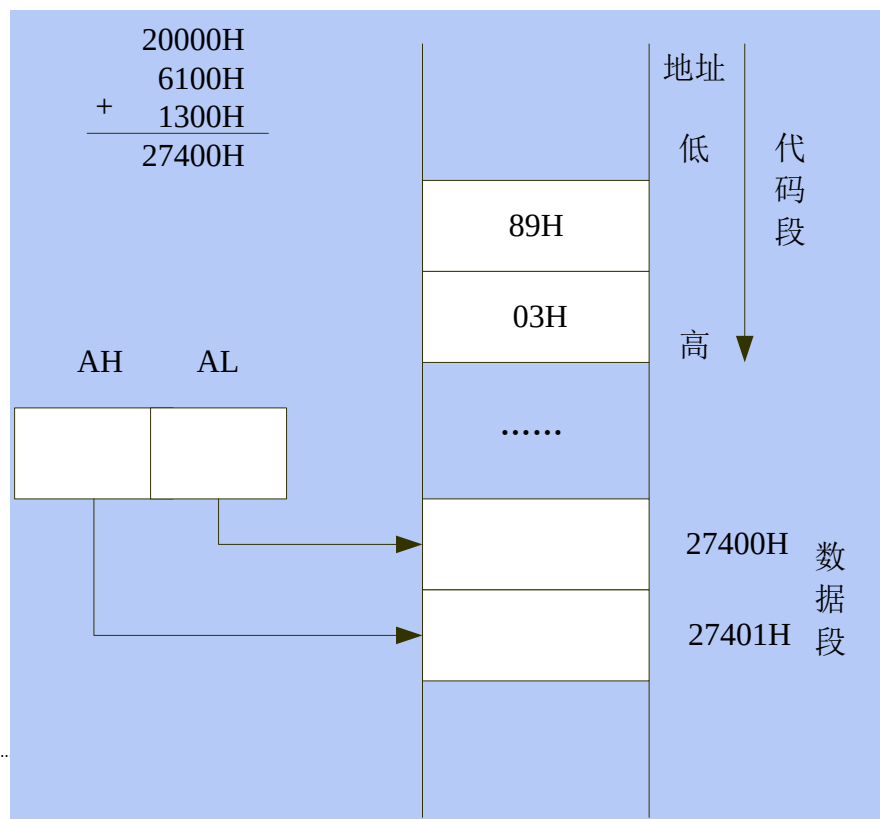
例 **MOV [BP+DI],AX;(SS)=2000H, (BP)=6100H, (DI)=1300H,**

目的操作数:基址变址寻址方式,

目的操作数的偏移地址=**(BP)+(DI)=7400H,**

目的操作数的段地址为**SS**的内容**2000H,**

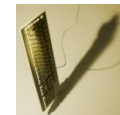
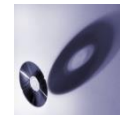
目的操作数的物理地址=**(SS) × 10H+(BP)+(DI)=27400H**



7) 相对基址变址寻址

操作数存放在存储器中，操作数的偏移地址是一个基址寄存器的内容加一个变址寄存器的内容再加一个**8位或16位**的位移量形成，即

$$EA = \begin{pmatrix} (SI) \\ (DI) \end{pmatrix} + \begin{pmatrix} (BX) \\ (BP) \end{pmatrix} + \begin{pmatrix} 8\text{位disp} \\ 18\text{位disp} \end{pmatrix}$$



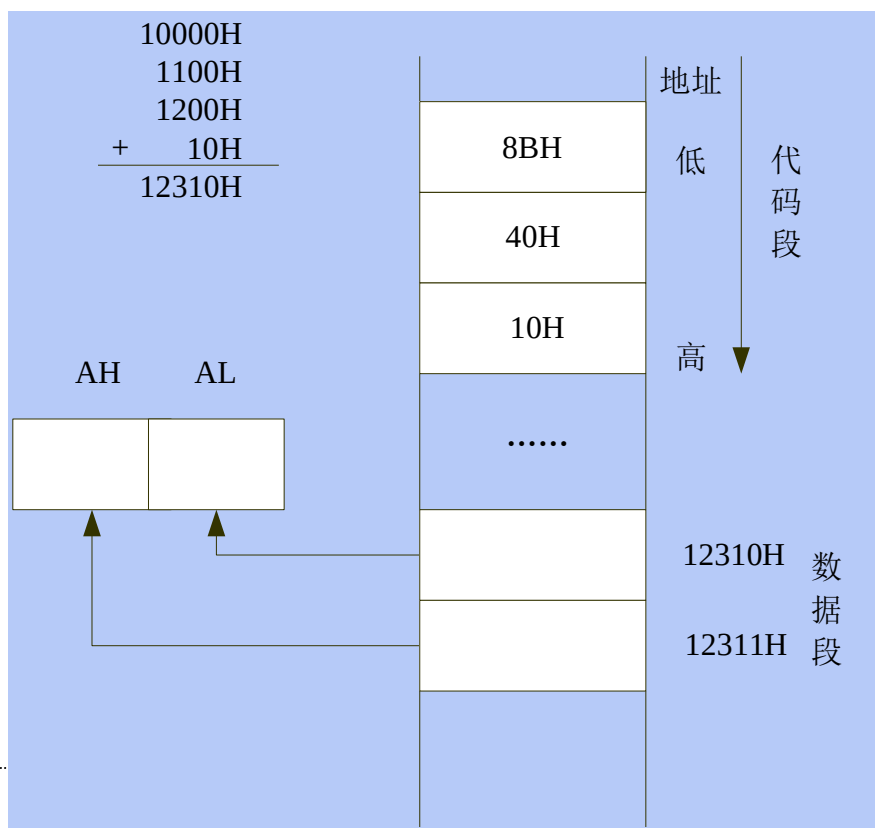
例 **MOV AX,[BX+SI+10H]** ;(DS)=1000H,(BX)=1100H,(SI)=1200H

源操作数:相对基址变址寻址,

源操作数的偏移地址=(BX)+(SI)+10H=2310H,

源操作数的段地址为DS的内容1000H,

源操作数的物理地址=(DS)×10H+(BX)+(SI)+10H=12310H



例 MOV [BP+DI+100H],AX

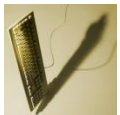
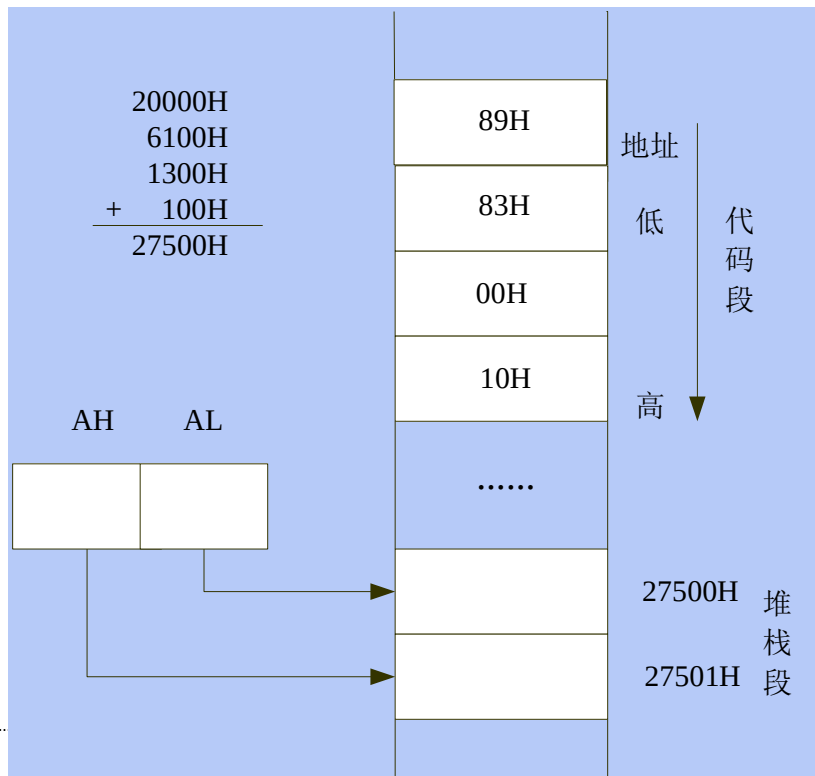
假设(SS)=2000H, (BP)=6100H, (DI)=1300H,

目的操作数:相对基址变址寻址方式,

目的操作数的偏移地址=(BP)+(DI)+100H=7500H,

目的操作数的段地址为SS的内容2000H,

目的操作数的物理地址=(SS)×10H+(BP)+(DI)+100H=27500H



按照操作数存放的位置，操作数可以分为立即数操作数，寄存器操作数和存储器操作数3类。

- **立即数操作数**：立即数存放在存储器的代码段，是指令的一部分。立即数操作数只能做为源操作数，不能做为目的操作数。
- **寄存器操作数**：寄存器操作数存放在CPU内部的寄存器中，指令执行时无需访问存储器，指令的执行速度快寄存器操作数在指令中既可以作为源操作数，也可以作为目的操作数。
- **存储器操作数**：直接寻址，寄存器间接寻址，寄存器相对寻址，基址变址寻址和相对基址变址寻址五种寻址方式对应的操作数为存储器操作数，存储器操作数存放在存储器中，其偏移地址通过寻址方式给出，其段地址在指令中隐含给出或者通过段超越前缀给出。存储器操作数在指令中可以作为源操作数或者目的操作数，但是8086CPU指令中，不允许两个操作数同时为存储器操作数。



练一练：写出下列指令中源操作数的寻址方式，如果是存储器操作数，写出其物理地址。已知：

(SS)=1500H , (DS)=2F00H , (BX)=0500H , (SI)=01F0H
(DI)=02E0H , (BP)=0500H , VOL=0100H

1 MOV AX , SI

1 寄存器寻址

2 MOV AX , 1500H

2 立即寻址

3 MOV AX , [BP]

3 寄存器间接寻址 15500H

4 MOV AX , [BX+SI]

4 基址—变址寻址 2F6F0H

5 MOV AX , [BX+VOL]

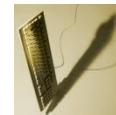
5 寄存器相对寻址 2F600H

6 MOV AX , [2100H]

6 直接寻址 31100H

7 MOV AX , [BP+DI+VOL]

相对的基址变址寻址 158E0H



3.3 指令系统

数据传送类

算术运算类

逻辑运算与移位类

字符串处理类

控制转移类

处理器控制类



指令的共性

A、可以实现字节操作，也可以实现字操作。

B、可以使用前面讲过的各种寻址方式，源操作数可以为立即数操作数，存储器操作数和寄存器操作数；目的操作数可以为寄存器操作数和存储器操作数。

C、两个操作数长度必须相同

D、当两个操作数为存储器操作数和立即数操作数时，必须指明内存单元的类型属性。内存单元的类型有3种：

DWORD PTR——双字类型，

WORD PTR——字类型，

BYTE PTR——字节类型。

E、两个操作数不能同时为存储器操作数。

F、**IP**不能作为操作数，**CS**不能做目的操作数。



3.3 指令系统

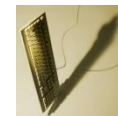
3.3.1 数据传送指令

1. 通用数据传送指令

1) 基本的传送指令MOV

指令一般形式: **MOV DST, SRC**

指令功能: **SRC**  **DST** (字节或字) ;



MOV指令可以实现如下数据传送

- MOV指令不能完成立即数到段寄存器的数据传送

MOV DS,1500H X

- 立即数操作数到存储器的传送

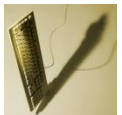
MOV [1000H],10H X

MOV BYTE PTR[1000H],10H

MOV WORD PTR[1000H],10H

- MOV指令不能实现段寄存器到段寄存器间的数据传送。

MOV SS,DS X



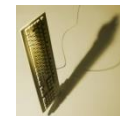
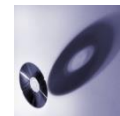
● 存储器与存储器之间的数据传送

注意：存储器与存储器之间的数据传送是不允许的，要实现数据从存储单元到存储单元的传送，必须借助**CPU**内部的寄存器作为“桥梁”来完成。

MOV [1000H],[BX] X

MOV AL,[BX]

MOV [1000H],AL



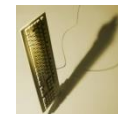
练一练：用一条传送指令完成：

- ① 给**AL**赋初值**69H**
- ② 将内存**2100H**字单元清零
- ③ 把内存以**BX**间接寻址的字节单元内容传送到**CL**中。

解：① **MOV AL, 69H**

② **MOV WORD PTR [2100H], 00H**

③ **MOV CL, [BX]**



2) 数据交换指令XCHG

格式: **XCHG OPRD1,OPRD2**

功能: 将操作数**OPRD1**和**OPRD2**进行交换, 即将操作数**OPRD1**送至操作数**OPRD2**, 同时将操作数**OPRD2**送至操作数**OPRD1**, 是一个双向数据传送, 可以实现字节数据或字数据的交换。

注意:

操作数**OPRD1**和**OPRD2**不能同时为存储器操作数

立即数操作数不能参与交换

段寄存器的内容不能参与交换



2.堆栈操作指令

1) 入栈指令PUSH

格式: **PUSH SRC**

功能: 将源操作数压入堆栈, 不影响标志位。

注意: **源操作数必须为字数据**。源操作数可以是寄存器操作数或者存储器操作数, 即源操作数可以是**16位**的通用寄存器, 段寄存器或者是**16位**的存储器操作数。

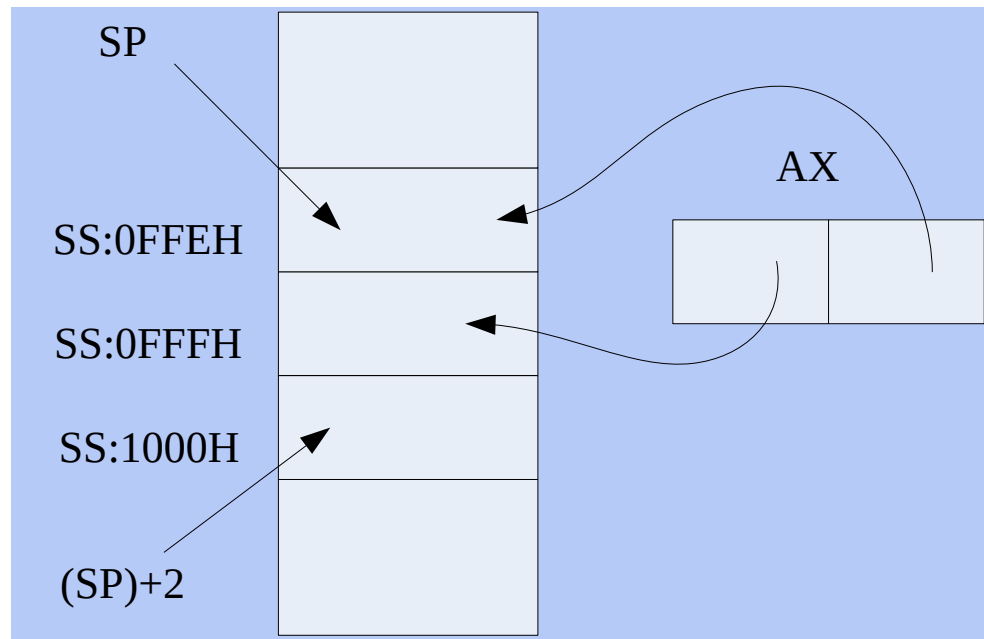
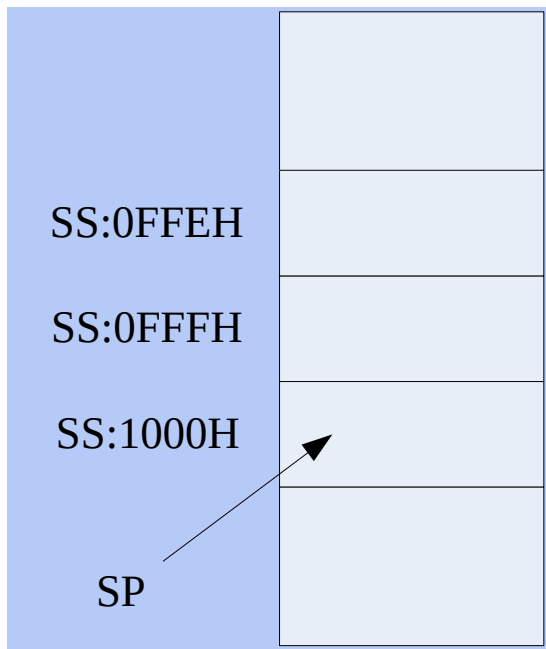
指令的执行过程:

修改堆栈指针**SP**, $(SP) - 2 \rightarrow (SP)$, 形成新的堆栈栈顶。
将源操作数送至新的堆栈栈顶存放。即源操作数的低**8位**送至**SP**指向的存储单元, 高**8位**送至 $(SP) + 1$ 指向的存储单元。



例: **PUSH AX**

假设指令执行前 $(AX) = 1122H$, $(SP) = 1000H$

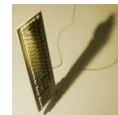
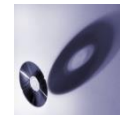


PUSH指令执行前堆栈段

PUSH指令执行后堆栈段

指令执行结果: $(SP) = 0FFE H$,

$(SS:0FFE H) = 22H$, $(SS:0FFF H) = 11H$



2)出栈指令POP

格式: **POP DST**

功能: 将当前堆栈栈顶的字数据弹出到目的操作数, 不影响标志位。

注意: 目的操作数的类型必须为字类型。目的操作数可以是寄存器操作数或者存储器操作数, 即源操作数可以是**16**位的通用寄存器, 段寄存器或者是**16**位的存储器操作数。

POP CS **X**

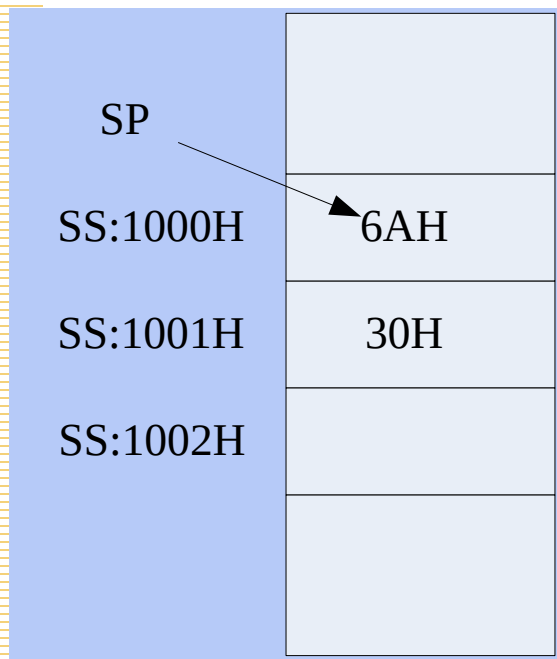
指令的执行过程:

将堆栈栈顶的字数据弹出到目的操作数给定的存储单元或者是寄存器, 即**SP**指向的存储单元的内容送至目的操作数的低**8**位, **(SP)+1**指向的存储单元的内容送至目的操作数的高**8**位。

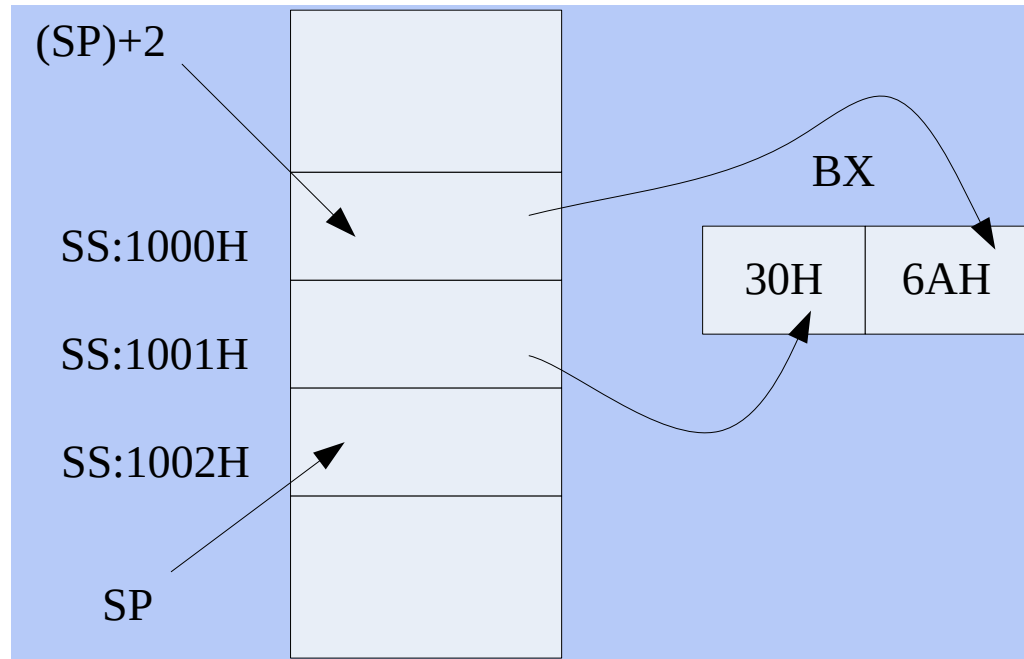
修改堆栈指针**SP**, $(SP) + 2 \rightarrow (SP)$, 形成新的堆栈栈顶。



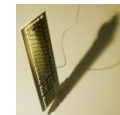
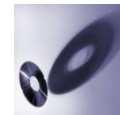
例: **POP BX** ; $((SP)) \rightarrow BL, ((SP) + 1) \rightarrow BH, (SP) + 2 \rightarrow SP$
假设指令执行前 $(BX) = 1122H$, $(SP) = 1000H$,
 $(SS: 1000H) = 6AH$, $(SS: 1001H) = 30H$



出栈指令执行前堆栈段



出栈指令执行后堆栈段



3.地址传送指令

1)取有效地址指令LEA

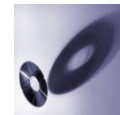
格式: **LEA REG16, MEM** ;

功能: 将源操作数的**16**位偏移地址传送至目的操作数。指令中要求源操作数必须是存储器操作数, 目的操作数必须是一个**16**位的寄存器操作数。

例: 已知(*DI*) = 0500H, (*DS:0610H*) = 75H, (*DS:0611H*) = 12H。

LEA BX, [DI+110H] ;指令执行后(*BX*) = 0610H

MOV BX, [DI+110H] ;指令执行后(*BX*) = 1275H



2)地址指针装入DS指令

格式: **LDS REG16, MEM32**

功能: 将源操作数地址所对应的**32**位存储器操作数送入到**DS**和指令中指定的寄存器。**32**位的存储器操作数是一个逻辑地址, 其高字部分为段地址, 低字部分为偏移地址, 指令执行时将**32**位存储器操作数的高字(段地址)送入到附加段寄存器**DS**, 低字(偏移地址)送入到目的操作数给出的**16**位寄存器。

例: **LDS BX, [1000H]**; 已知**(DS)=1200H**

指令执行结果: **(DS)=2000H, (BX)=5018H**。

13000H	18H
13001H	50H
13002H	00H
13003H	20H



3)地址指针装入ES指令

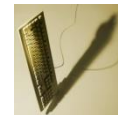
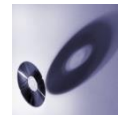
格式: **LES REG16, MEM32**

功能: 将源操作数地址所对应的**32**位存储器操作数送入到**ES**和指令中指定的寄存器。**32**位的存储器操作数是一个逻辑地址, 其高字部分为段地址, 低字部分为偏移地址, 指令执行时将**32**位存储器操作数的高字(段地址)送入到附加段寄存器**ES**, 低字(偏移地址)送入到目的操作数给出的**16**位寄存器。

例: **LES SI, [BX]** ;将**BX**指向的存储单元中的字数据送入**BX**,
;将**(BX)+2**指向的存储单元中的字数据送入**ES**

已知**(DS)=2000H**, **(BX)=2000H**, 存储单元**(22000H)=0A0H**,
(22001H)=10H, **(22002H)=00H**, **(22003H)=18H**。

指令执行结果: **(ES)=1800H**, **(BX)=10A0H**。



4.输入输出指令

输入输出指令实现的是**CPU与I/O端口**之间的数据传送。

输入指令**IN**: I/O端口——→ 累加器(**AL或AX**),

输出指令**OUT**:累加器(**AL或AX**) ——→ I/O端口。

I/O端口的寻址方式(I/O端口的地址范围: 0000H~FFFFH)

- **直接寻址**: 在指令中直接给出**I/O端口的地址**对**I/O端口**进行访问。只有部分**I/O端口**可以直接访问, 其地址范围为**00H~FFH**。
- **间接寻址**: 在访问**I/O端口**时, 不直接给出**I/O端口的地址**, 而是将**I/O端口的地址**送入到**DX寄存器**, 通过**DX寄存器**间接访问**I/O端口**。所有的**I/O端口**都可以采用间接寻址方式进行访问。



1) 输入指令IN

格式: **IN ACC, PORT**
IN ACC, DX

功能: 将端口中的**8位**或**16位**数据输入至累加器**AL**或**AX**。

例: **IN AL, 80H**

MOV DX, 2100H
IN AL, DX



2) 输出指令OUT

格式: **OUT PORT, ACC**
OUT DX, ACC

功能: 将累加器**AL**或**AX**中的**8**位或**16**位数据输出至**I/O**端口

例: **OUT 78H, AL**
MOV DX, 183H
OUT DX, AL



例：编程序段，将内存**1000H**的一个字节数据传送到外设端口地址为**62CH**中。

```
MOV AL, [1000H]
```

```
MOV DX, 62CH
```

```
OUT DX, AL
```

例：编程序段，将外设端口地址为**62CH**中的一个字节数据传送到内存**1000H**单元。

端口地址为**62H**时，编写程序段

```
MOV DX, 62CH
```

```
IN AL, DX
```

```
MOV [1000H], AL
```

```
IN AL, 62H
```

```
MOV [1000H], AL
```



6)查表指令XLAT

格式: **XLAT** ;((**BX**) + (**AL**)) → **AL**

功能: 完成一个字节的查表转换。要求表格的长度小于**256B**。

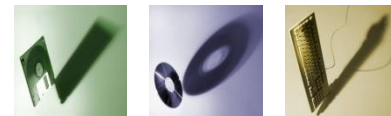
例如: 利用**XLAT**指令查找数字**0~9**的共阴极七段数码管码值。
TABEL DB 3FH,06H,5BH,4FH,66H,6DH,7DH,07H,7FH,6FH
TABEL表中存放了共阴极七段数码管数字**0~9**的段码值, 使用查表指令查找数字**7**的段码值并送累加器**AL**。

程序代码如下:

LEA BX,TABEL

MOV AL,7

XLAT



3.3.2 算术运算指令

1. 加法运算指令

1) 不带进位的加法指令

格式: **ADD DST, SRC** ;(*DST*) + (*SRC*) → *DST*

功能: (*DST*) + (*SRC*) → *DST*, 按照执行结果影响全部的状态标志位**CF**、**SF**、**OF**、**ZF**、**PF**、**AF**。

例:

MOV AL,76H

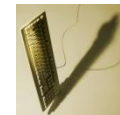
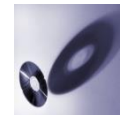
MOV BL,45H

ADD AL,BL

$$\begin{array}{r} 01110110 \\ + 01000101 \\ \hline 10111011 \end{array}$$

执行结果: (AL)=0BBH

SF=1 ZF=0 OF=1 PF=1 AF=0 CF=0



2)带进位的加法指令ADC(Add with Carry)

格式: **ADC DST, SRC** ;(*DST*) + (*SRC*) + CF → DST

功能: (*DST*) + (*SRC*) + CF → DST, 按照执行结果影响全部的状态标志位**CF**、**SF**、**OF**、**ZF**、**PF**、**AF**。**ADC**指令适用于多字节数据的加法运算。



3)加1指令INC(Increment)

格式: **INC DST**

功能: **$DST + 1 \rightarrow DST$** **INC**指令**不影响进位标志CF**, 按照运算结果影响标志位**PF, AF, SF, ZF和OF**。

INC指令中目的操作数可以是**8位或16位**的寄存器操作数或者存储器操作数, 不能是立即数操作数, 并且当操作数为存储器操作数时, 需要规定存储器操作数的类型。



例：已知内存数据区DATA1和DATA2开始的区域存放有5字节数据3B74AC60F8H和20D59E36C1H，要求编程求两个多字节数据的和并保存至DATA1开始的存储器。

程序代码如下：

```
MOV CX, 5
```

```
MOV SI, 0
```

```
CLC
```

```
AGAIN:
```

```
MOV AL, DATA2[SI]
```

```
ADC DATA1[SI], AL
```

```
INC SI
```

```
DEC CX
```

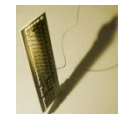
```
JNZ AGAIN
```

DATA1

F8H	
60H	
ACH	
74H	
3BH	

DATA2

C1H
36H
9EH
D5H
20H



2.减法运算指令

1)不带借位的减法指令SUB(Subtraction)

格式: **SUB DST, SRC**

功能: **(DST) - (SRC) → DST** 按照操作的结果影响标志位 **CF、SF、OF、ZF、PF、AF**。

例:

MOV AL,76H

MOV BL,45H 0100 0101B

SUB AL,BL

$$\begin{array}{r} 01110110 \\ +10111011 \\ \hline 00110001 \end{array}$$

执行结果: **(AL)=31H**

SF=0 ZF=0 OF=0 PF=0 AF=0 CF=0



2)带借位的减法指令SBB(Subtraction with Borrow)

格式: SBB DST, SRC

功能: $(DST) - (SRC) - CF \rightarrow DST$, 同时按照运行结果影响标志位**CF**、**SF**、**OF**、**ZF**、**PF**、**AF**。SBB指令一般用于多字节减法运算中。

例: 在8086系统中, 实现32位整数(双字数据)的减法运算,

被减数: **DX: AX** 减数: **BX: CX**

差: **DX: AX**

(DX)=1A24H (AX)=E308H (BX)=730DH (CX)=28E3H

分析: **DX:AX**

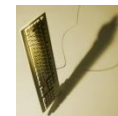
- BX: CX

DX:AX

程序代码

SUB AX, CX

SBB DX, BX



3)减1指令DEC(Decrement)

格式: **DEC DST**

功能: $(DST) - 1 \rightarrow DST$, **DEC**指令**不影响进位标志CF**, 按照运算结果影响标志位**PF, AF, SF, ZF和OF**。

DEC指令中目的操作数可以是**8位或16位**的寄存器操作数或者存储器操作数, 不能是立即数操作数, 并且当操作数为存储器操作数时, 需要规定存储器操作数的类型。

DEC, INC指令一般在循环程序中用于修改地址指针或循环次数。



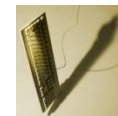
4)求补指令NEG(Negative)

格式: **NEG DST**

功能: **$0 - (DST) \rightarrow DST$** ,

NEG指令中目的操作数可以是**8位或16位**的寄存器操作数或者存储器操作数, 不能是立即数操作数。当操作数为存储器操作数时, 需要规定存储器操作数的类型。

NEG指令一般可以对负数求绝对值。



5)比较指令CMP(Compare)

格式: **CMP DST, SRC**

功能: **(DST) – (SRC)**, 按照差值影响标志位**CF**、**SF**、**OF**、**ZF**、**PF**、**AF**, 利用标志位比较两个操作数的大小。

例: 在内存数据段从**DATA**开始的两个单元分别存放了两个**8**位无符号数, 试比较它们的大小, 并将其中较大的数送至数据区**1000H**单元。

```
LEA BX,DATA
MOV AL,[BX]
INC BX
CMP AL,[BX]
JNC NEXT
MOV AL,[BX]
NEXT: MOV [1000H],AL
```



3.乘法运算指令

无符号数乘法指令格式:

MUL SRC

; (SRC) $\times$ (AL) $\rightarrow$ (AX) 字节乘法

; (SRC) $\times$ (AX) $\rightarrow$ DX:AX 字乘法

带符号数乘法指令格式:

IMUL SRC

; (SRC) $\times$ (AL) $\rightarrow$ (AX) 字节乘法

; (SRC) $\times$ (AX) $\rightarrow$ DX:AX 字乘法



注意：

1指令中的源操作数只能是**寄存器操作数**或者**存储器操作数**，不能为立即数操作数，当操作数为存储器操作数时，需要规定存储单元的属性。

2乘法指令影响标志位**CF**、**OF**。

无符号乘法MUL对CF、OF的影响：

乘积的高半部分(字节乘法时为AH，字乘法时为DX)为0，则**CF=OF=0**；

乘积的高半部分(字节乘法时为AH，字乘法时为DX)不为0，**CF=OF=1**。

带符号乘法IMUL对CF、OF的影响：乘积的高半部分仅仅是符号位的扩展，即AH=0或0FFH，DX=0或0FFFFH，则**CF=OF=0**；

如果乘积的高半部分不仅仅是符号位的扩展，包含有效数字，即AH $\neq$ 0且AH $\neq$ 0FFH或DX $\neq$ 0且DX $\neq$ 0FFFFH，则**CF=OF=1**。



例：

MOV AL,87H

MOV BL,35H

MUL BL

执行结果：(AX)=1BF3H，CF=OF=1 表明乘积AH中包含了有效数字。

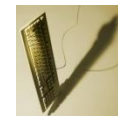
例：

MOV AX, 35E3H

MOV BX, 2487H

IMUL BX

执行结果：(DX)=07B0H，(AX)=56B5H，CF=OF=1，表明DX中包含有效数字，而不仅仅是符号位的扩展。



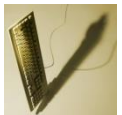
4.除法运算指令

无符号数除法指令:

DIV SRC ;(AX)/(SRC) → AL 字节除法
 ;(AX)%(SRC) → AH
 ;(DX:AX)/(SRC) → AX 字除法
 ;(DX:AX)%(SRC) → DX

带符号数除法指令:

IDIV SRC ;除操作对象为带符号数外，其余同DIV



注意：

1指令中的源操作数只能是**寄存器操作数**或者**存储器操作数**，不能为立即数操作数，当操作数为存储器操作数时，需要规定存储单元的属性。

2除法指令对标志位的影响无定义。

3除法指令中要求**被除数为除数长度的2倍**，在使用除法指令进行除法运算时，如果被除数和除数长度相同，需要使用符号扩展指令对被除数的长度进行扩展。

4在使用除法运算指令时，必须考虑除法的**溢出**问题。商超出**AL(AX)**表示范围时发生溢出。

5在执行带符号数除法指令时，余数的符号与被除数相同。



例：

MOV AX,87

MOV CL,10

DIV CL

执行结果： (AL)=8 (AH)=7

例：

MOV AX,-87

MOV CL,10

IDIV CL

执行结果： (AL)=0F8H(-8) (AH)=0F9H(-7)



5.符号扩展指令

- 字节扩展成字指令**CBW(Convert Byte to Word)**

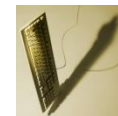
格式: **CBW**

;将**AL**中的数据符号位扩展到**AH**中, 形成字数据(**AX**)

- 字扩展成双字指令**CWD(Convert Word to Double Word)**

格式: **CWD**

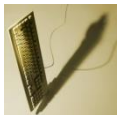
;将**AX**中数据的符号位扩展到**DX**中, 形成双字数据(**DX:AX**)



例：内存数据段存放了**10**个带符号数，数据的首地址为**DATA1**，将数据取绝对值后存入以**DATA2**为首地址的内存区。

程序代码：

```
LEA SI, DATA1
LEA DI, DATA2
MOV CX, 10
CHECK: MOV AL, [SI]
CMP AL, 0
JGE POSZERO
NEG AL
POSZERO: MOV [DI], AL
INC SI
INC DI
LOOP CHECK
HLT
```



3.3.3 逻辑运算与移位指令

1. 逻辑运算指令

1) 逻辑非指令 NOT

格式: **NOT DST** ; $0FFFFH - DST \rightarrow DST$ 字运算

; $0FFH - DST \rightarrow DST$ 字节运算

功能: 目的操作数的每一位按位取反后, 结果送目的操作数。

例:

MOV AL, 0FH

NOT AL

执行结果: $(AL) = 0F0H$



2)逻辑与指令AND

格式: **AND DST, SRC**

功能: **$(DST) \wedge (SRC) \rightarrow DST$** , 按照运算结果影响标志位**ZF、SF和PF**, **CF和OF清为0**, 对**AF**的影响未定义。

逻辑与运算指令通常用于对目的操作数的**某一位或多位进行清0**, 其他位保留不变, 进行清0处理的位与**0**进行逻辑与操作, 保留位与**1**进行逻辑与操作。

例:

MOV AL,0FFH

AND AL, 0FH

对**AL**的高4位进行清0操作, 低4位保留不变。

执行结果: **(AL)=0FH**, **CF=0**, **OF=0**, **ZF=0**, **PF=1**, **SF=0**



3)逻辑或指令OR

格式: OR DST, SRC

功能: $(DST) \vee (SRC) \rightarrow DST$ 按照运算结果影响标志位ZF、SF和PF, CF和OF清为0, 对AF的影响未定义。

逻辑或运算指令通常用于对目的操作数的某一位或多位进行置位操作, 其他位保留不变, 进行置位的位与1进行逻辑或运算, 保留位与0进行逻辑或运算。

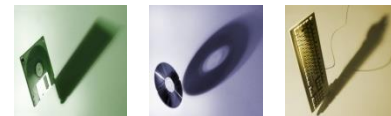
例:

MOV AX, 1038H

OR AX, 2300H 0010 0011 0000 0000B

对AX中的D₈, D₉, D₁₃位进行了置位操作, 其余位保留不变。

执行结果: (AX)=3338H, CF=0, OF=0, ZF=0, PF=0, SF=0



4)逻辑异或指令XOR

格式: XOR DST, SRC

功能: $(DST) \oplus (SRC) \rightarrow DST$, 按照运算结果影响标志位ZF、SF和PF, CF和OF清为0, 对AF的影响未定义。

逻辑异或运算指令通常用于对目的操作数的某一位或多位进行取反操作, 其他位保留不变, 进行取反的位与1进行逻辑异或运算, 保留位与0进行逻辑异或运算。

例:

MOV AL, 55H

XOR AL, 0FFH 1111 1111B

对AL中的每一位都进行了取反操作。

执行结果: (AL)=0AAH, CF=0, OF=0, ZF=0, PF=1, SF=1



5)逻辑测试指令TEST

格式: **TEST DST, SRC**

功能: **$(DST) \wedge (SRC)$** , 但是按照运算结果影响标志位**ZF**、**SF**和**PF**, **CF**和**OF**清为**0**, 对**AF**的影响未定义。

逻辑测试运算指令通常用于测试目的操作数的某一位是否为**1**, 测试方法是将被测试位与**1**进行逻辑测试运算, 其余位与**0**进行逻辑测试运算, 根据标志位**ZF**或**PF**判定被测试位的值。

ZF=1 $\longrightarrow$ 被测试位为**0**

ZF=0 $\longrightarrow$ 被测试位为**1**

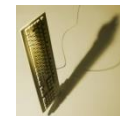
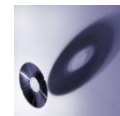
对于最高位的测试也可以通过**SF**标志位的值进行判断。

例: 检测**CL**寄存器中数据的**D₂**位

TEST CL, 4 ;0000 0**1**00B

方法: **CL**中的**D₂**和**1**进行测试操作, 其余位与**0**进行测试操作。

判别方法: 如果**ZF = 1**则**D₂=0**; 如果**ZF=0**, 则**D₂=1**



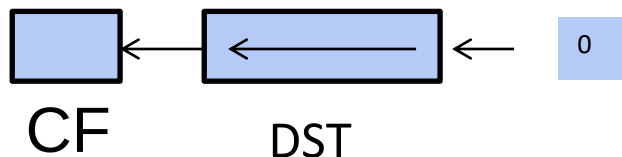
2.移位类指令

(1)一般移位类指令

1)逻辑/算术左移指令SHL/SAL(Shift logic left/shift arithmetic left)

格式: **SHL/SAL DST,1/CL**

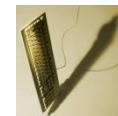
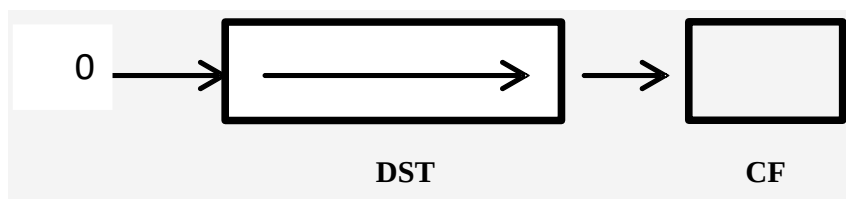
功能: 将目的操作数逻辑左移/算术左移一位或者**CL**寄存器中指定的位数。
逻辑/算术移位指令左移一位时, 目的操作数的每一位左移一位, 最高位进入进位标志**CF**, 最低位补**0**。



2)逻辑右移指令SHR(shift logical right)

格式: **SHR DST,1/CL**

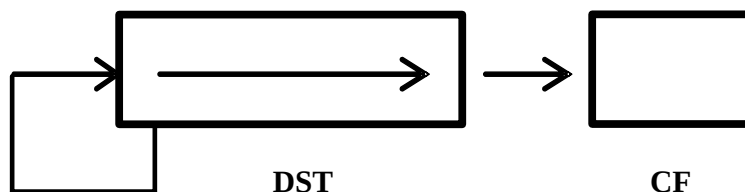
功能: 将目的操作数逻辑右移一位或者**CL**寄存器中指定的位数。
逻辑右移一位时, 目的操作数的每一位右移一位, 最低位进入进位标志**CF**, 最高位补**0**。



3)算术右移指令SAR(shift arithmetic right)

格式: **SAR DST,1/CL**

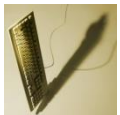
功能: 将目的操作数算术右移一位或者**CL**寄存器中指定的位数。
算术右移一位时, 目的操作数的每一位右移一位, 最低位进入进位标志**CF**, 最高位补原来的最高位。



- 源操作数必须为1或CL，用于指明移位的位数

- 左移相当于乘法运算，左移1位相当于乘以2
右移相当于除法运算，右移1位相当于除以2
算术移位的操作对象为带符号数
逻辑移位的操作对象为无符号数

- 对标志位的影响
按照结果影响SF，PF和ZF
对CF的影响：总是等于最后移入CF的值
对OF的影响：移1位时由影响
右移：移位结束最高位和次高位 相同 OF=0 不同OF=1
左移：移位结束最高位和CF 相同OF=0 不同OF=1

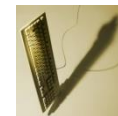
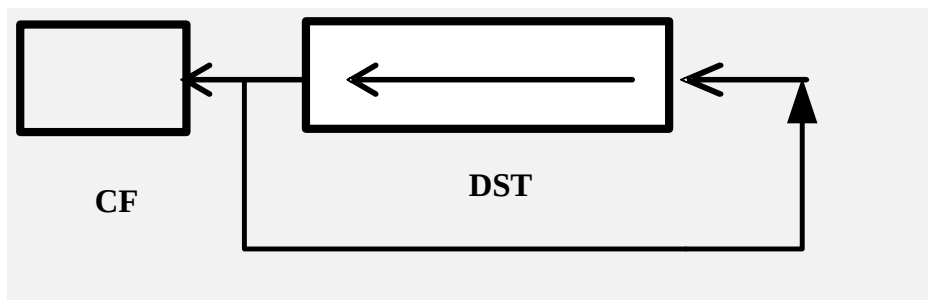


(2)循环移位指令

1)循环左移指令ROL(rotate left)和 循环右移指令ROR(rotate right)

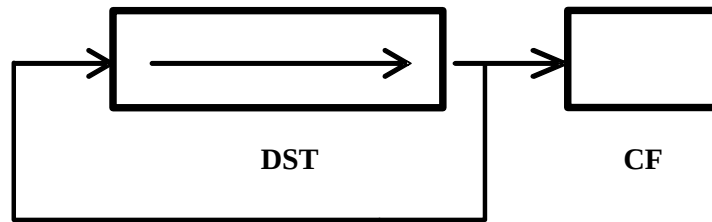
循环左移指令格式：**ROL DST,1/CL**

功能：将目的操作数循环左移一位或者左移**CL**寄存器指定的位数，最高位移至**CF**，最低位补原来操作数的最高位，目的操作数本身形成一个封闭循环。

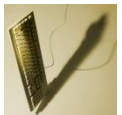


循环右移指令格式: **ROR DST,1/CL**

功能: 将目的操作数循环右移一位或者左移**CL**寄存器指定的位数, 最低位移至**CF**, 最高位补原来操作数的最低位, 目的操作数本身形成一个封闭循环。



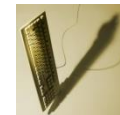
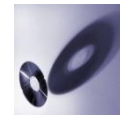
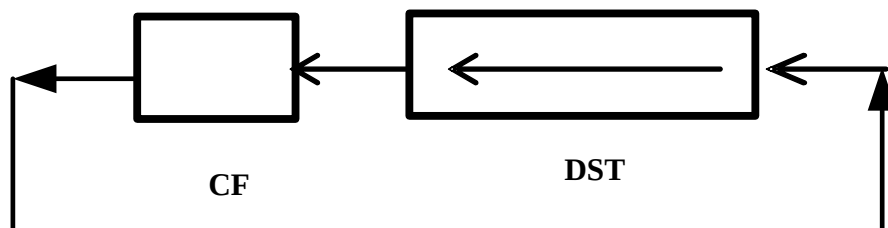
注意: 循环移位指令影响标志位**CF**和**OF**。当循环左移指令移位位数为一位时, 移位完成后目的操作数的最高位和**CF**相等, 则**OF=0**, 否则**OF=1**。当循环右移指令移位位数为一位时, 移位完成后目的操作数的最高位和次高位相等, 则**OF=0**, 否则**OF=1**。**CF**总是等于最后移入到**CF**的值。



2)带进位循环左移指令RCL(rotate left through carry)和带进位循环右移指令RCR(rotate right through carry)

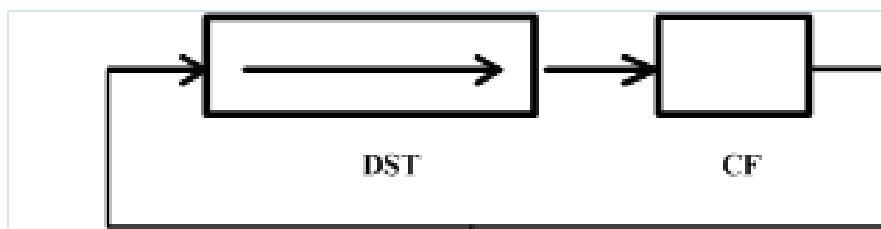
带进位循环左移指令RCL格式: **RCL DST,1/CL**

功能: 将目的操作数循环左移一位或者左移CL寄存器指定的位数, 最高位移至CF, 最低位补指令执行前CF的值, 目的操作数和CF共同形成一个封闭循环。

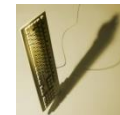
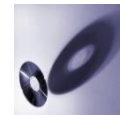


带进位循环右移指令**RCR**格式: **RCR DST,1/CL**

功能: 将目的操作数循环右移一位或者左移**CL**寄存器指定的位数, 最低位移至**CF**, 最高位补指令执行前**CF**的值, 目的操作数和**CF**共同形成一个封闭循环。

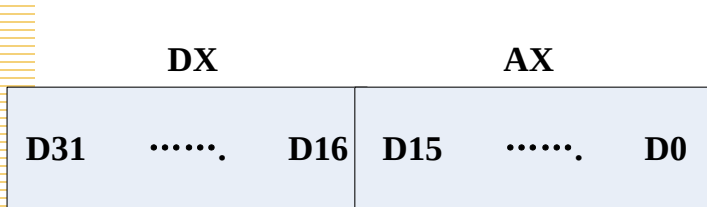


注意: 带进位循环移位指令影响标志位**CF**和**OF**。当循环左移指令移位位数为一位时, 移位完成后目的操作数的最高位和**CF**相等, 则**OF=0**, 否则**OF=1**。当循环右移指令移位位数为一位时, 移位完成后目的操作数的最高位和次高位相等, 则**OF=0**, 否则**OF=1**。**CF**总是等于最后移入到**CF**的值。

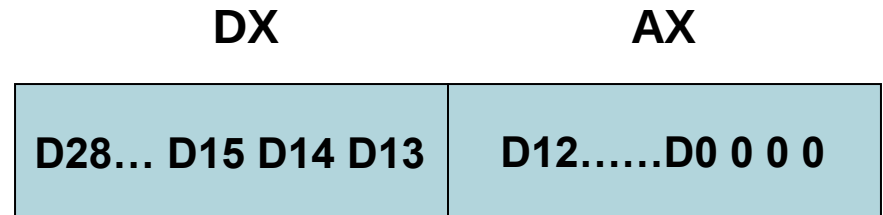


例：将存放于**DX:AX**中**32**位数乘以**8**。

分析：**32**位数据的低**16**位**D₀~D₁₅**存放在**AX**中，高**16**位**D₁₆~D₃₁**存放在**DX**中



a 原始数据



b 处理后数据

程序代码：

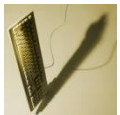
MOV CX, 3

AGAIN: SAL AX, 1

RCL DX, 1

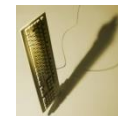
DEC CX

JNZ AGAIN



3.3.5 控制转移类指令

- 转移指令
- 循环指令
- 子程序调用和返回指令
- 中断指令和中断返回指令



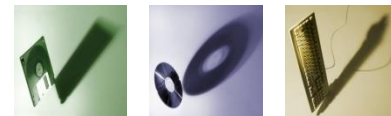
1.转移指令

(1) 无条件转移指令JMP

格式: **JMP DST**

功能: 程序在不需要满足任何条件的情况下转移到目的操作数指定的地址去执行指令。

注意: 目的操作数可以是**标号**, **立即数操作数**, 也可以是**寄存器操作数**或者**存储器操作数**。



1) 段内直接转移

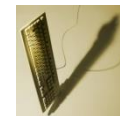
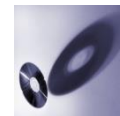
转移目标地址在转移指令的操作数部分直接给出，并且转移目标地址和转移指令地址位于同一个逻辑段，称为段内直接转移。

A. 段内直接短转移

格式： **JMP SHORT LABEL**

功能：程序无条件转移到**LABEL**标号处执行程序。

段内直接短转移是指目标指令的地址与转移指令地址之间的相对偏移量范围在**-128~127**之间，此时目标指令的标号是一个短标号，用关键字**SHORT**标示。



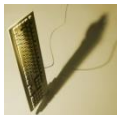
B.段内直接近转移

格式: **JMP NEAR PTR LABEL**

JMP 偏移地址

功能: 程序无条件转移到**LABEL**标号处执行程序或者程序转移到指令中给定的偏移地址处执行。

段内直接近转移是指目标指令的地址与转移指令地址之间的相对偏移量范围在**-32768~32767**之间。指令汇编后, 计算出目标指令地址和转移指令之间的**16**位相对偏移量, 指令执行时, 将当前**IP**值加上一个**16**位偏移量形成新的**IP**值, 实现程序的转移。



2) 段内间接转移

格式: **JMP REG16**
JMP MEM16

操作数说明: 指令中的操作数可以是**16**位的寄存器操作数(寄存器寻址)或者**16**位的存储器操作数(除立即寻址和寄存器寻址以外的其他寻址方式)。

功能: 程序无条件转移到操作数指定的目标地址。



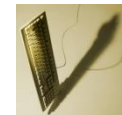
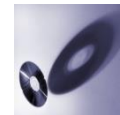
3)段间直接转移

格式: **JMP FAR PTR LABEL**

JMP 段地址: 偏移地址

功能: 指令无条件转移到**LABEL**标号处执行, **LABEL**标号是一个属性为**FAR**的标号, 或者程序转移到指令指定的逻辑地址处执行。

执行的操作: 将标号的偏移地址送至**IP**, 将标号的的段地址送至**CS**, 或者将指令中给定的偏移地址送至**IP**, 段地址送至**CS**, 使程序的执行顺序发生改变。

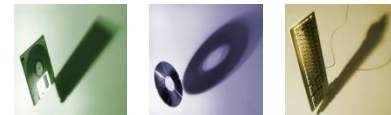


4)段间间接转移

格式: **JMP MEM32**

功能: 指令无条件转移到操作数指定的地址执行。

执行的操作: 指令的操作数是一个**32**位的存储器操作数, 其高字部分为段地址, 低字部分为偏移地址, 将操作数低字送至**IP**, 将操作数高字送至**CS**, 使程序的执行顺序发生改变。



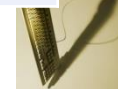
(2)条件转移指令

1)根据两个操作数比较形成的条件转移指令

助记符	测试条件	转移条件	备注
JA/JNBE	CF=0且ZF=0	高于/不低于等于转移	无符号数
JAE/JNB	CF=0	高于等于/不低于转移	
JB/JNAE	CF=1	低于/不高于等于转移	
JBE/JNA	CF=1或ZF=1	低于等于/不高于转移	
JG/JNLE	SF+OF=0且ZF=0	大于/不小于等于转移	带符号数
JGE/JNL	SF+OF=0	大于等于/不小于转移	
JL/JNGE	SF+OF=1且ZF=0	小于/不大于等于转移	
JLE/JNG	SF+OF=1	小于等于/不大于转移	

2)根据一个标志位形成的条件转移指令

助记符	测试条件	转移条件
JZ/JE	ZF=1	结果为0或相等转移
JNZ/JNE	ZF=0	结果不为0或不相等转移
JC	CF=1	有进位时转移
JNC	CF=0	无进位时转移
JS	SF=1	结果为负时转移
JNS	SF=0	结果为非负时转移
JP/JPE	PF=1	结果中1的个数为偶数时转移
JNP	PF=0	结果中1的个数为奇数时转移
JO	OF=1	结果溢出时转移
JNO	OF=0	结果不溢出时转移



例：已知寄存器**AX**和**DX**中各存放了一个带符号字数据，现要求将较大的字数据存放在**AX**中，较小者存放在**DX**中。

```
CMP AX,DX  
JGE NEXT  
MOV BX,AX  
MOV AX,DX  
MOV DX,BX  
NEXT: HLT
```



例：已知内存数据区自**2000H**开始存放着**10**个带符号字节数据，统计其中正数，负数和零的个数，分别放入**BL**，**DL**和**DH**中。

CHECK:

```
MOV SI,2000H
MOV DX,0
MOV BL,0
MOV CX,0
```

```
MOV AL,[SI]
AND AL,AL
```

```
JS NEG
```

```
JZ ZERO
```

```
INC BL
```

```
JMP NEXT
```

NEG:

```
INC DL
```

```
JMP NEXT
```

ZERO:

```
INC DH
```

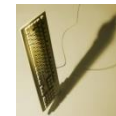
NEXT:

```
INC SI
```

```
DEC CX
```

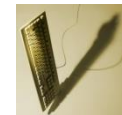
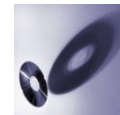
```
JNZ CHECK
```

```
HLT
```

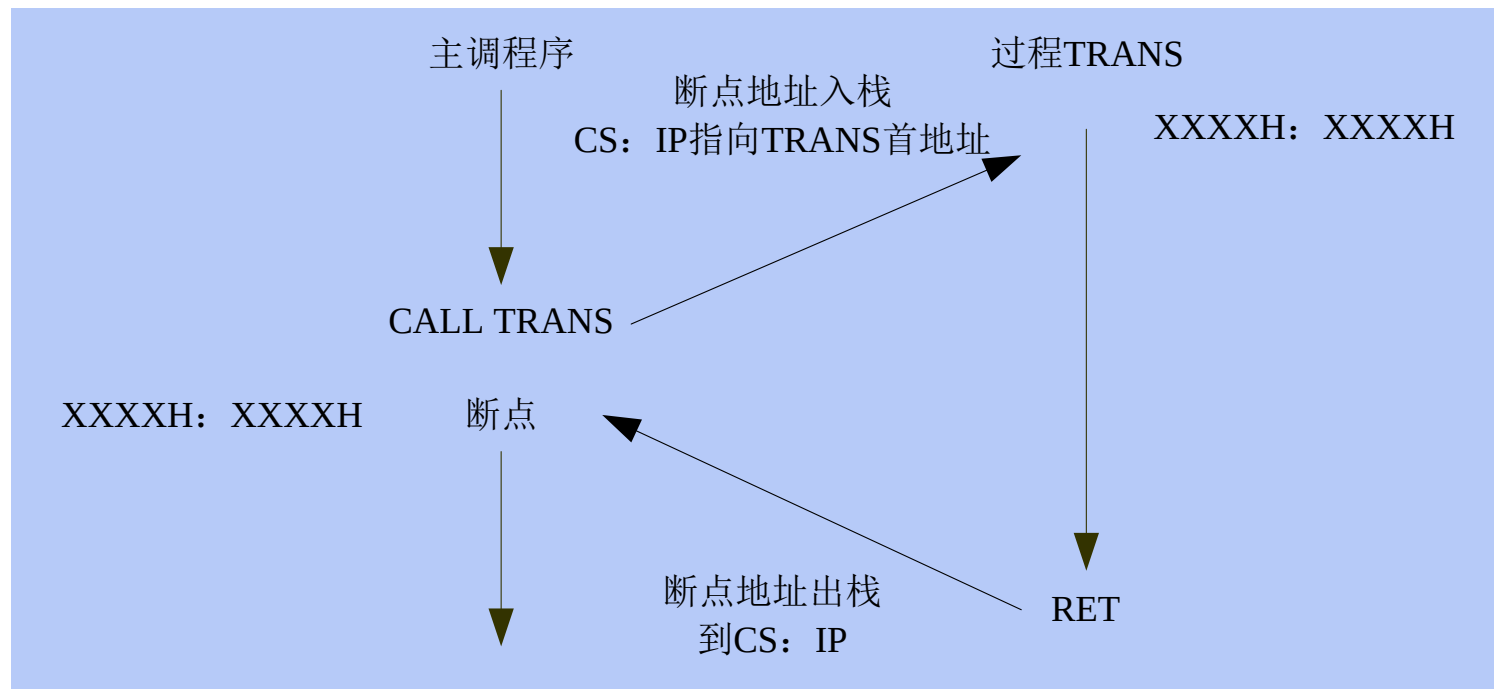


2. 循环转移指令(和CX有关)

指令助记符	测试条件	指令功能
LOOP	$CX \neq 0$	$(CX)-1 \rightarrow (CX)$, $CX \neq 0$ 转移到标号
LOOPZ/LOOPE	$CX \neq 0$ 且 $ZF=1$	$(CX)-1 \rightarrow (CX)$, $CX \neq 0$ 且 $ZF=1$ 转移到标号
LOOPNZ/LOOPNE	$CX \neq 0$ 且 $ZF=0$	$(CX)-1 \rightarrow (CX)$, $CX \neq 0$ 且 $ZF=0$ 转移到标号
JCXZ	$CX \neq 0$	$CX \neq 0$ 转移到标号



3.过程调用和返回指令



过程调用和返回执行过程示意图



1.过程调用指令CALL(Call Procedure)

(1)段内直接调用

格式: **CALL Near_Proc** ;(SP)-2→(SP), (IP)→((SP)),
(IP)+16disp→(IP)

功能: 调用过程Near_Proc。

指令执行过程: 指令的操作数是一个近过程, 和调用指令**CALL**在同一个逻辑段。指令经过汇编后, 将得到断点与过程入口地址之间的**16位位移量16disp**, 首先将断点偏移地址(即**IP**的值)压入堆栈, 然后将**IP**加上计算出的**16位位移量**, 实现从主调程序到过程的切换。

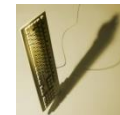
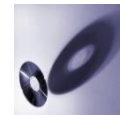


(2)段内间接调用

格式: **CALL REG16/MEM16 ;(SP)-2→ (SP), (IP)→ ((SP)), (REG16)/(MEM16) (IP)**

功能: 调用寄存器操作数或者存储器操作数指向的过程。

指令执行过程: 指令的操作数是一个**16**位的寄存器操作数或**16**位的存储器操作数, 寄存器或存储器的值是过程的入口地址, 过程和调用指令**CALL**在同一个逻辑段。指令执行时, 先将断点偏移地址(即**IP**的值)压入堆栈, 然后将寄存器或者存储器的值传送到**IP**, 实现从主调程序到过程的切换。



(3)段间直接调用

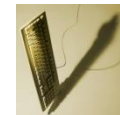
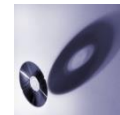
格式: **CALL Far_Proc**

;**(SP)-2→(SP), (CS)→((SP)), SEG Far_Proc→(CS)**

;**(SP)-2→(SP) (IP)→((SP)), OFFSET Far_Proc→(IP)**

功能: 调用过程**Far_Proc**。

指令执行过程: 指令的操作数是一个远过程, 和调用指令**CALL**不在同一个逻辑段。指令执行时, 首先将断点段地址(即**CS**的值)压入堆栈, 并将远过程的段地址送入**CS**, 然后将断点偏移地址(即**IP**的值)压入堆栈, 将远过程首指令的偏移地址送入**IP**, 实现从主调程序到过程的切换。



(4)段间间接调用

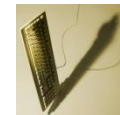
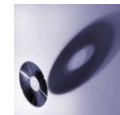
格式: **CALL MEM32**

; (SP)-2 → (SP), (CS) → ((SP)), (MEM32+2) → (CS)

; (SP)-2 → (SP) (IP) → ((SP)), (MEM32) → (IP)

功能: 调用存储器操作数指向的过程。

指令执行过程: 指令的操作数是一个**32位**存储器操作数指向的过程, 和调用指令**CALL**不在同一个逻辑段。指令执行时, 首先将断点段地址(即**CS**的值)压入堆栈, 并将**32位**存储器操作数的高字(过程的段地址)送入**CS**, 然后将断点偏移地址(即**IP**的值)压入堆栈, 将**32位**存储器操作数的低字(即过程的偏移地址)送入**IP**, 实现从主调程序到过程的切换。



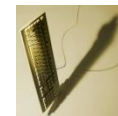
2)返回指令RET(Return from a Procedure)

1)段内返回

格式1: **RET** ;(SP)→(IP), (SP)+2→(SP)

格式2: **RET n** ;(SP)→(IP), (SP)+2→(SP), (SP)+n→(SP)

当调用的过程为近过程时，返回指令为段内返回。对于格式1的返回，其功能是将栈顶的内容(断点偏移地址)送至IP，然后堆栈指针加2；格式2的返回为带参数的返回，其功能是先将栈顶的内容(断点偏移地址)送至IP，然后堆栈指针加2，最后堆栈指针再加n，即舍弃堆栈中的n个字节数据，这些字节数据一般是通过堆栈向过程传递的参数。其中n是一个范围在0~64KB的偶数。

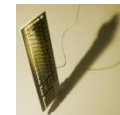
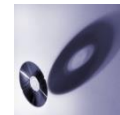


2)段间返回

格式1: **RET** ;(**SP**)→(**IP**), (**SP**)+2→(**SP**)
 ;(**SP**)→(**CS**), (**SP**)+2→(**SP**)

格式2: **RET n** ;(**SP**)→(**IP**), (**SP**)+2→(**SP**)
 ;(**SP**)→(**CS**), (**SP**)+2→(**SP**), (**SP**)+**n**→(**SP**)

当调用的过程为远过程时，返回指令为段间返回。对于格式1的返回，其功能是将栈顶的内容(断点偏移地址)送至**IP**，然后堆栈指针加2；接着将栈顶的内容(断点段地址)送至**CS**，堆栈指针加2。格式2的返回为带参数的返回，其功能是先将栈顶的内容(断点偏移地址)送至**IP**，然后堆栈指针加2，接着将栈顶的内容(断点段地址)送至**CS**，堆栈指针加2。最后堆栈指针再加**n**，**n**和段内返回**RET n**中的**n**含义相同。



4.中断指令和中断返回指令

中断指令INT(Interrupt)

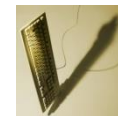
格式: INT n

$;(SP)-2 \rightarrow (SP), (FLAGS) \rightarrow (SP), 0 \rightarrow (IF), 0 \rightarrow (TF)$

$;(SP)-2 \rightarrow (SP), (CS) \rightarrow (SP), (n \times 4 + 2) \rightarrow (CS)$

$;(SP)-2 \rightarrow (SP), (IP) \rightarrow (SP), (n \times 4) \rightarrow (IP)$

功能: 转去中断类型号n对应的中断服务程序。



2) 溢出中断指令INTO

格式: **INTO** ;无操作数, 隐含的n值为4

功能: 检测**OF**, 如果**OF=1**, 启动中断类型为4的中断服务程序; 如果**OF=0**, 不执行任何操作。

3) 中断返回指令IRET

中断返回指令**IRET**是中断服务程序最后一条可执行指令。

格式: **IRET** ;(**SP**) $\rightarrow$ (**IP**), (**SP**)+2 $\rightarrow$ (**SP**)
;(**SP**) $\rightarrow$ (**CS**), (**SP**)+2 $\rightarrow$ (**SP**)
;(**SP**) $\rightarrow$ (**FLAGS**), (**SP**)+2 $\rightarrow$ (**SP**)

功能: 结束中断, 从中断服务程序返回断点。



3.3.6 处理器控制指令

类		助记符	指令功能
标志控制	进位标志CF	CLC STC CMC	进位标志清0 进位标志置位 进位标志取反
	方向标志DF	CLD STD	方向标志清0 方向标志置位
	中断标志IF	CLI STI	中断标志清0 中断标志置位
外部同步指令		WAIT	处理器等待，与外部硬件同步
		ESC	处理器交权，将控制权交给其他处理器
		LOCK	总线锁定，禁止其他处理器访问总线
		HLT	处理器暂停
空操作		NOP	CPU不执行任何操作，IP改变